

JWT AUTH — THIS REPO'S VERSION

The generic two-token story, replayed with this app's real cookies, endpoints and expiries

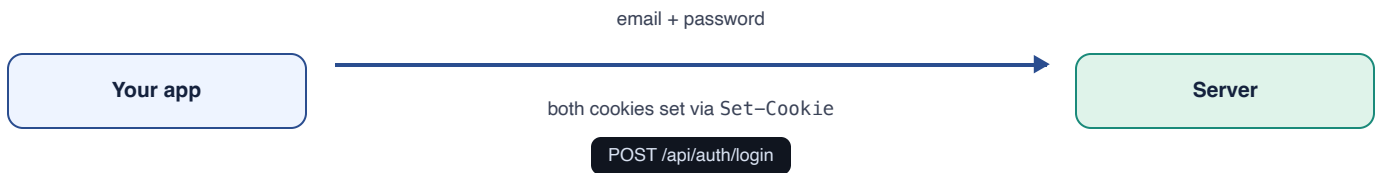
1. Login → 2. Normal request → 3. Token expires → 4. Silent refresh → 5. Retry → Logout

★ The one big difference from the generic pattern

The reference deck describes the access token living in a **JS variable your React code holds**, sent as an `Authorization: Bearer` header. **This repo does not do that.** Both tokens here are `httpOnly` cookies — page JS never reads, holds, or sends either token itself. The browser attaches them automatically because every request is same-origin (via the Vite proxy). This is a stricter anti-XSS posture than the generic pattern, at the cost of needing `SameSite/cookie` config instead of a header.

1 The two tokens, this repo's actual values

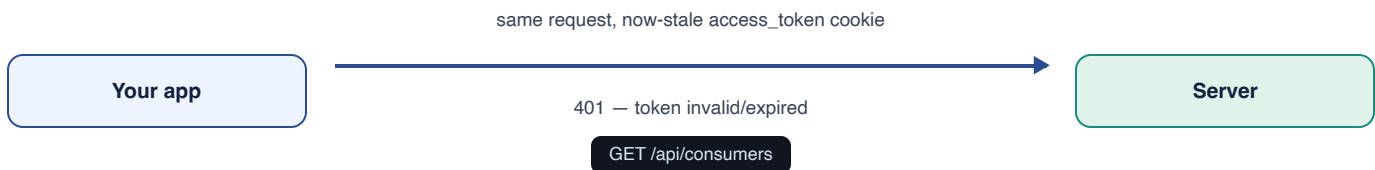
	ACCESS_TOKEN	REFRESH_TOKEN
Lifetime	15 minutes	30 days
Where it lives	httpOnly cookie, SameSite=Lax, Path=/ — both, not just one	
Sent on	Every request (browser attaches automatically)	Only POST /api/auth/refresh
Signed with	JWT_ACCESS_SECRET	JWT_REFRESH_SECRET — a different secret, so a leaked access secret can't forge a refresh token
Payload	{ sub: userId, email }	{ sub: userId } only — email is re-fetched from Supabase on refresh, never trusted stale

1 You log in

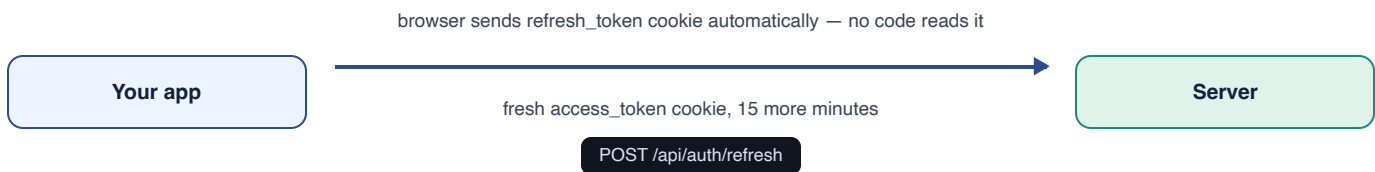
- `bcrypt.compare` against `demo_users.password_hash` in Supabase. Wrong password and unknown email get the **identical** generic error — the response can't be used to check which emails are registered.
- The response body is `{ user: { id, email } }` — never the tokens themselves. Only the `Set-Cookie` headers carry them.

2 The app asks for data — the normal case

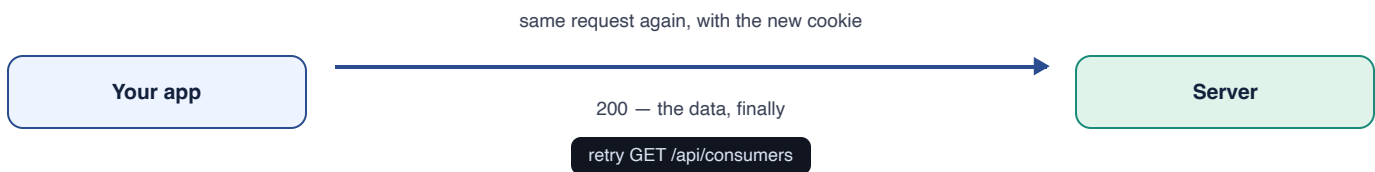
- `verifyAccessToken()` checks the JWT signature **locally** with `jwt.verify` — no database round-trip, no network call to anywhere. This is what "hand-rolled" auth means in this repo (vs. a delegated provider).

3 Fifteen minutes later, it expires

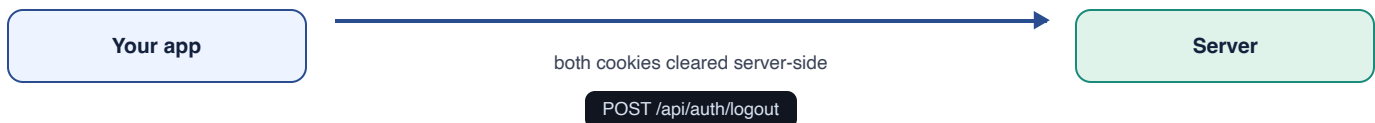
- Nothing is broken and the user did nothing wrong — the token simply aged out, exactly as designed. Every protected route in `server/routes/consumers.js` behaves the same way via its shared `requireAuth` middleware.

4 The app quietly gets a new access token

- Server re-fetches the user row from Supabase by the sub (user id) in the refresh token — never trusts a cached email from the old token.
- If the refresh token is itself invalid/expired, this returns 401 too — there is no third token to fall back on. That's the actual end of the session.

5 The app repeats the request that failed

- Steps 3, 4 and 5 all happen automatically inside the axios response interceptor in `client/src/lib/axios.ts` — the user sees a half-second delay, not a login screen. Full mechanics in `AXIOS-INTERCEPTORS-EXPLAINED.pdf`.

Logging out is the mirror image of logging in

- `res.clearCookie(...)` for both — there's no token block-list to check against, because a cleared cookie simply never gets sent again.

See also: `JWT-AUTH-FROM-SCRATCH.md` (the full concept walkthrough, incl. `bcrypt` and `httpOnly` rationale), `AXIOS-INTERCEPTORS-EXPLAINED.pdf` (the client-side half of steps 3–5), and milestones M1–M11 in `AUTH-LEARNING-MILESTONES.md`.